

BÀI TẬP THỰC HÀNH SỐ 4

Học phần: Nền tảng phát triển Web – Dành cho: K65

Xây dựng một ứng dụng Quản Lý Sách (Book Manager)

Mục tiêu

- Tạo ứng dụng React đơn giản với **Create React App** (hoặc Vite).
- Quản lý danh sách sách (books) bằng **state** trong React.
- Thực hiện các thao tác **thêm, hiển thị, sửa, xóa** sách.
- (Tùy chọn) Lưu dữ liệu vào **Local Storage** để không mất khi tải lại trang.

Nội dung & Phân bổ thời gian (100 phút)

Thời gian	Hoạt động	Chi tiết thực hiện
0-5'	Giới thiệu bài	Nêu mục tiêu, tổng quan về CRUD và cấu trúc ứng dụng React
5-15'	Bước 1: Khởi tạo dự án	Tạo dự án, xóa file mẫu, cấu trúc thư mục cơ bản
15-30'	Bước 2: Hiển thị danh sách sách	Tạo component <code>BookList</code> , hiển thị data mẫu, giải thích state/props
30-50'	Bước 3: Thêm sách (Create)	Tạo form <code>BookForm</code> để thêm sách, cập nhật state, hiển thị lại danh sách
50-70'	Bước 4: Chỉnh sửa sách (Update)	Chọn sách cần sửa, hiện form có sẵn dữ liệu, cập nhật state khi submit
70-85'	Bước 5: Xóa sách (Delete)	Thêm nút “Xóa” trong danh sách, remove khỏi state
85-95'	Bước 6: Lưu Local Storage (tùy chọn)	Lưu/đọc state từ <code>localStorage</code> , đồng bộ khi CRUD
95-100'	Tổng kết & Q&A	Chạy thử, kiểm tra, hỏi – đáp, hướng dẫn cải tiến (pagination, search, ...)

Hướng dẫn chi tiết

Dưới đây là các bước với code mẫu để sinh viên có thể làm theo. Có thể dùng **Create React App** hoặc **Vite** (tùy ý).

Bước 1: Khởi tạo dự án (5-15')

1. **Tạo dự án** (dùng Create React App):
2. `npx create-react-app book-manager`
3. `cd book-manager`
4. `npm start`

Hoặc với **Vite**:

```
npm create vite@latest book-manager -- --template react
cd book-manager
npm install
npm run dev
```

5. **Xóa bớt file mẫu** (logo, test, v.v.) để dự án gọn nhẹ.
6. **Kiểm tra chạy thành công** ở `http://localhost:3000/` (hoặc `http://127.0.0.1:5173/` với Vite).

Cấu trúc thư mục gợi ý:

```
book-manager/
├── public/
├── src/
│   ├── components/
│   │   ├── BookList.js
│   │   └── BookForm.js
│   ├── App.js
│   └── index.js
└── package.json
...
```

Bước 2: Hiện thị danh sách sách (15-30')

1. **Khởi tạo dữ liệu mẫu**: Trong `App.js`, tạo state `books` chứa một mảng sách.
2. **Tạo component** `BookList.js` để hiện thị danh sách.

Ví dụ code:

App.js:

```
import React, { useState } from 'react';
import BookList from './components/BookList';

function App() {
  // Dữ liệu mẫu ban đầu
  const [books, setBooks] = useState([
    { id: 1, title: 'React Cơ Bản', author: 'Nguyễn Văn A', year: 2023 },
    { id: 2, title: 'JavaScript Nâng Cao', author: 'Trần Thị B', year: 2022 },
  ]);

  return (
    <div style={{ margin: '20px' }}>
      <h1>Quản Lý Sách</h1>
      {/* Truyền books xuống BookList qua props */}
      <BookList books={books} />
    </div>
  );
}
```

```

    </div>
  );
}

export default App;

```

BookList.js:

```

import React from 'react';

function BookList({ books }) {
  return (
    <div>
      <h2>Danh sách sách</h2>
      <ul>
        {books.map(book => (
          <li key={book.id}>
            <strong>{book.title}</strong> - {book.author} ({book.year})
          </li>
        ))}
      </ul>
    </div>
  );
}

export default BookList;

```

Bước 3: Thêm sách (Create) (30-50')

1. **Tạo form** để nhập thông tin sách: `BookForm.js`.
2. **Xử lý sự kiện** submit form, cập nhật state `books` trong `App.js`.

BookForm.js (ví dụ):

```

import React, { useState } from 'react';

function BookForm({ onAdd }) {
  const [title, setTitle] = useState('');
  const [author, setAuthor] = useState('');
  const [year, setYear] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title || !author || !year) {
      alert('Vui lòng nhập đầy đủ thông tin!');
      return;
    }
    // Tạo object sách mới
    const newBook = {
      id: Date.now(), // Tạo ID tạm
      title,
      author,
      year: parseInt(year, 10),
    };
    onAdd(newBook);

    // Xóa trắng form
    setTitle('');
    setAuthor('');
    setYear('');
  };

  return (
    <form onSubmit={handleSubmit} style={{ marginBottom: '20px' }}>
      <h2>Thêm Sách</h2>

```

```

    <div>
      <label>Tiêu đề: </label>
      <input
        value={title}
        onChange={ (e) => setTitle(e.target.value) }
      />
    </div>
    <div>
      <label>Tác giả: </label>
      <input
        value={author}
        onChange={ (e) => setAuthor(e.target.value) }
      />
    </div>
    <div>
      <label>Năm XB: </label>
      <input
        type="number"
        value={year}
        onChange={ (e) => setYear(e.target.value) }
      />
    </div>
    <button type="submit">Thêm</button>
  </form>
);
}

export default BookForm;

```

Chỉnh sửa App.js để nhúng BookForm và xử lý thêm sách:

```

import React, { useState } from 'react';
import BookList from './components/BookList';
import BookForm from './components/BookForm';

function App() {
  const [books, setBooks] = useState([
    { id: 1, title: 'React Cơ Bản', author: 'Nguyễn Văn A', year: 2023 },
    { id: 2, title: 'JavaScript Nâng Cao', author: 'Trần Thị B', year: 2022 },
  ]);

  // Hàm thêm sách
  const handleAddBook = (newBook) => {
    setBooks([...books, newBook]);
  };

  return (
    <div style={{ margin: '20px' }}>
      <h1>Quản Lý Sách</h1>
      <BookForm onAdd={handleAddBook} />
      <BookList books={books} />
    </div>
  );
}

export default App;

```

- Giảng viên giải thích **useState** cho form, cách **onSubmit** gọi hàm `onAdd`.

Bước 4: Chỉnh sửa sách (Update) (50-70')

1. Tạo nút “Sửa” cho mỗi sách trong `BookList`.
2. Khi bấm “Sửa”, ta lấy thông tin sách và đổ vào `BookForm` hoặc 1 form khác.
3. Cập nhật state khi người dùng xác nhận chỉnh sửa.

Ý tưởng 1: Dùng **cùng 1 form** `BookForm` để thêm + sửa (dựa vào biến `editingBook`).

Ý tưởng 2: Tạo **component riêng** `EditBookForm`.

Dưới đây là ví dụ ý tưởng 1 (dùng chung form):

- Trong `App.js`, thêm state `editingBook` (null nếu không sửa).
- Nếu người dùng bấm “Sửa”, set `editingBook = book` cần sửa.
- `BookForm` nhận thêm prop `bookToEdit` và `onUpdate(book)`.
- Nếu `bookToEdit` có giá trị, form sẽ hiển thị dữ liệu để người dùng chỉnh.

App.js (tóm tắt ý chính):

```
function App() {
  const [books, setBooks] = useState([...]);
  const [editingBook, setEditingBook] = useState(null);

  // Hàm thêm
  const handleAddBook = (newBook) => {
    setBooks([...books, newBook]);
  };

  // Hàm cập nhật
  const handleUpdateBook = (updatedBook) => {
    const newList = books.map(b =>
      b.id === updatedBook.id ? updatedBook : b
    );
    setBooks(newList);
    setEditingBook(null); // Thoát chế độ sửa
  };

  // Khi bấm nút "Sửa" trong BookList
  const handleEditClick = (book) => {
    setEditingBook(book);
  };

  return (
    <div style={{ margin: '20px' }}>
      <h1>Quản Lý Sách</h1>
      <BookForm
        onAdd={handleAddBook}
        onUpdate={handleUpdateBook}
        editingBook={editingBook}
      />
      <BookList
        books={books}
        onEdit={handleEditClick}
      />
    </div>
  );
}
```

BookList.js (thêm nút Sửa):

```
function BookList({ books, onEdit }) {
  return (
    <div>
      <h2>Danh sách sách</h2>
      <ul>
        {books.map(book => (
          <li key={book.id}>
            <strong>{book.title}</strong> - {book.author} ({book.year})
            { ' ' }
            <button onClick={() => onEdit(book)}>Sửa</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```

    </ul>
  </div>
);
}

```

BookForm.js (thêm logic cho chế độ sửa):

```

function BookForm({ onAdd, onUpdate, editingBook }) {
  const [title, setTitle] = useState('');
  const [author, setAuthor] = useState('');
  const [year, setYear] = useState('');

  // Mỗi khi editingBook thay đổi, nạp dữ liệu vào form
  useEffect(() => {
    if (editingBook) {
      setTitle(editingBook.title);
      setAuthor(editingBook.author);
      setYear(editingBook.year);
    } else {
      setTitle('');
      setAuthor('');
      setYear('');
    }
  }, [editingBook]);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title || !author || !year) {
      alert('Vui lòng nhập đầy đủ!');
      return;
    }

    if (editingBook) {
      // Update
      onUpdate({
        ...editingBook,
        title,
        author,
        year: parseInt(year, 10),
      });
    } else {
      // Add
      const newBook = {
        id: Date.now(),
        title,
        author,
        year: parseInt(year, 10),
      };
      onAdd(newBook);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>{editingBook ? 'Sửa Sách' : 'Thêm Sách'}</h2>
      { /* ...các input... */ }
      <button type="submit">
        {editingBook ? 'Cập nhật' : 'Thêm'}
      </button>
    </form>
  );
}

```

- Giảng viên giải thích **useEffect** để nạp dữ liệu khi `editingBook` thay đổi, logic phân nhánh **Add / Update**.

Bước 5: Xóa sách (Delete) (70-85')

1. Thêm nút “Xóa” bên cạnh “Sửa” trong `BookList`.
2. Khi bấm, gọi hàm `handleDeleteBook(id)` trong `App.js` để xóa khỏi state.

Ví dụ:

BookList.js:

```
<li key={book.id}>
  <strong>{book.title}</strong> - {book.author} ({book.year})
  {' '}
  <button onClick={() => onEdit(book)}>Sửa</button>
  {' '}
  <button onClick={() => onDelete(book.id)}>Xóa</button>
</li>
```

App.js:

```
const handleDeleteBook = (id) => {
  const newList = books.filter(b => b.id !== id);
  setBooks(newList);
};

return (
  <BookList
    books={books}
    onEdit={handleEditClick}
    onDelete={handleDeleteBook}
  />
);
```

- Giảng viên nhấn mạnh cách filter mảng để xóa item.

Bước 6: Lưu Local Storage (85-95') (tùy chọn)

1. **Load** dữ liệu từ `localStorage` khi khởi tạo app.
2. **Lưu** lại mỗi khi `books` thay đổi.

Ví dụ:

```
// App.js
import React, { useState, useEffect } from 'react';
// ...

function App() {
  const [books, setBooks] = useState([]);

  // Load từ localStorage khi component mount
  useEffect(() => {
    const stored = localStorage.getItem('books');
    if (stored) {
      setBooks(JSON.parse(stored));
    }
  }, []);

  // Lưu mỗi khi books thay đổi
  useEffect(() => {
    localStorage.setItem('books', JSON.stringify(books));
  }, [books]);
}
```

```
// ...các hàm CRUD, return JSX...  
}
```

- Giảng viên giải thích cách sử dụng **useEffect** để đồng bộ books và localStorage.

Bước 7: Tổng kết & Q&A (95-100')

- Chạy thử ứng dụng, kiểm tra thêm, sửa, xóa sách.
 - Xem dữ liệu đã được lưu (nếu có Local Storage).
 - Giải đáp câu hỏi của sinh viên.
 - Gợi ý **cải tiến**:
 - Thêm tính năng **tìm kiếm** (search).
 - Tách code gọn gàng hơn (tạo `EditBookForm.js` riêng).
 - Tạo **bảng** đẹp hơn bằng **CSS** hoặc **Bootstrap**.
 - Triển khai API giả (json-server) để **luyện tập fetch**.
-